

//Queue STL operations

```
#include <iostream>
#include <queue>
using namespace std;

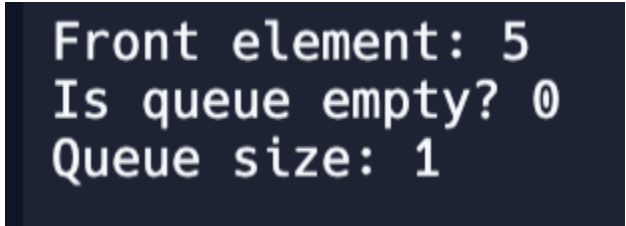
int main(){
    queue<int> Q;

    Q.push(5);
    Q.push(3);

    cout<< "Front element: " << Q.front() << endl;
    Q.pop();

    cout << "Is queue empty? " << Q.empty() << endl;
    cout << "Queue size: " << Q.size() << endl;

    return 0;
}
```



```
Front element: 5
Is queue empty? 0
Queue size: 1
```

//implementing a queue using a singly linked list

```
#include <iostream>
using namespace std;

struct Node{
    int data;
    Node* next;
};

class Queue {
private:
    Node* frontNode;
    Node* backNode;
    int count;
public:
    Queue() : frontNode(nullptr), backNode(nullptr), count(0) {}

    void push(int val){
```

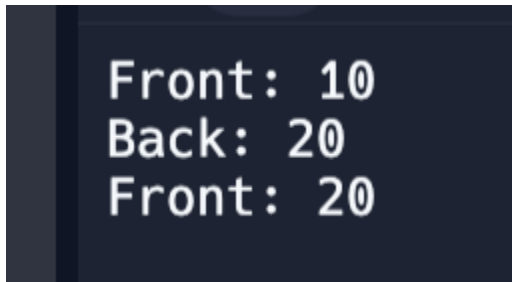
```

Node* newNode = new Node;
newNode->data = val;
newNode->next = nullptr;
if(backNode == nullptr){
    frontNode = backNode = newNode;
} else {
    backNode->next = newNode;
    backNode = newNode;
}
count++;
}
void pop(){
    if (frontNode == nullptr){
        cout << "Queue is empty" << endl;
        return;
    } else {
        Node* temp = frontNode;
        frontNode = frontNode->next;
        if (frontNode == nullptr){
            backNode = nullptr;
            delete temp;
        }
        count--;
    }
}
int front(){
    if (frontNode != nullptr){
        return frontNode->data;
    } else{
        cout << "Queue is empty" << endl;
        return -1;
    }
}
int back(){
    if (backNode != nullptr){
        return backNode-> data;
    } else{
        cout << "Queue is empty" << endl;
        return -1;
    }
}
int size() {return count;}
bool empty() {return frontNode == nullptr;}

```

```
};

int main(){
    Queue q;
    q.push(10);
    q.push(20);
    cout << "Front: " << q.front() << endl;
    cout<< "Back: " << q.back() << endl;
    q.pop();
    cout <<"Front: " << q.front() << endl;
    return 0;
}
```



```
Front: 10
Back: 20
Front: 20
```

//Queue with array

```
#include <iostream>
using namespace std;
```

```
class Queue {
private:
    int* arr;
    int capacity;
    int frontIndex;
    int backIndex;
    int count;

public:
    Queue(int size = 10){
        arr = new int[size];
        capacity = size;
        frontIndex = 0;
        backIndex = -1;
        count = 0;
    }
    ~Queue(){
        delete[] arr;
    }
    void push(int val){
```

```

    if (size() == capacity){
        cout << "Queue is full" << endl;
        return;
    }
    backIndex = (backIndex + 1) % capacity;
    arr[backIndex] = val;
    count++;
}

```

```

void pop() {
    if (empty()){
        cout << "Queue is empty" << endl;
        return;
    }
    frontIndex = (frontIndex + 1) % capacity;
    count--;
}

```

```

int front() {
    if (!empty()){
        return arr[frontIndex];
    } else {
        cout << "Queue is empty" << endl;
        return -1;
    }
}

```

```

int back(){
    if (!empty()){
        return arr[backIndex];
    } else {
        cout << "Queue is empty" << endl;
        return -1;
    }
}

```

```

int size(){
    return count;
}

```

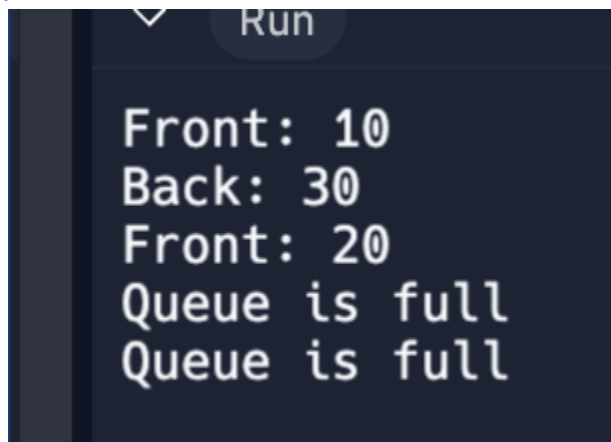
```

bool empty(){
    return (count == 0);
}

```

```
};
```

```
int main(){  
    Queue q(5);  
    q.push(10);  
    q.push(20);  
    q.push(30);  
    cout << "Front: " << q.front() << endl;  
    cout << "Back: " << q.back() << endl;  
    q.pop();  
    cout << "Front: " << q.front() << endl;  
    q.push(40);  
    q.push(50);  
    q.push(60);  
    q.push(70);  
    q.push(80);  
    return 0;  
}
```

A screenshot of a terminal window with a dark background. At the top, there is a button labeled 'Run' with a downward-pointing arrow. Below the button, the output of the program is displayed in white text. The output consists of five lines: 'Front: 10', 'Back: 30', 'Front: 20', 'Queue is full', and 'Queue is full'.

```
Run  
Front: 10  
Back: 30  
Front: 20  
Queue is full  
Queue is full
```